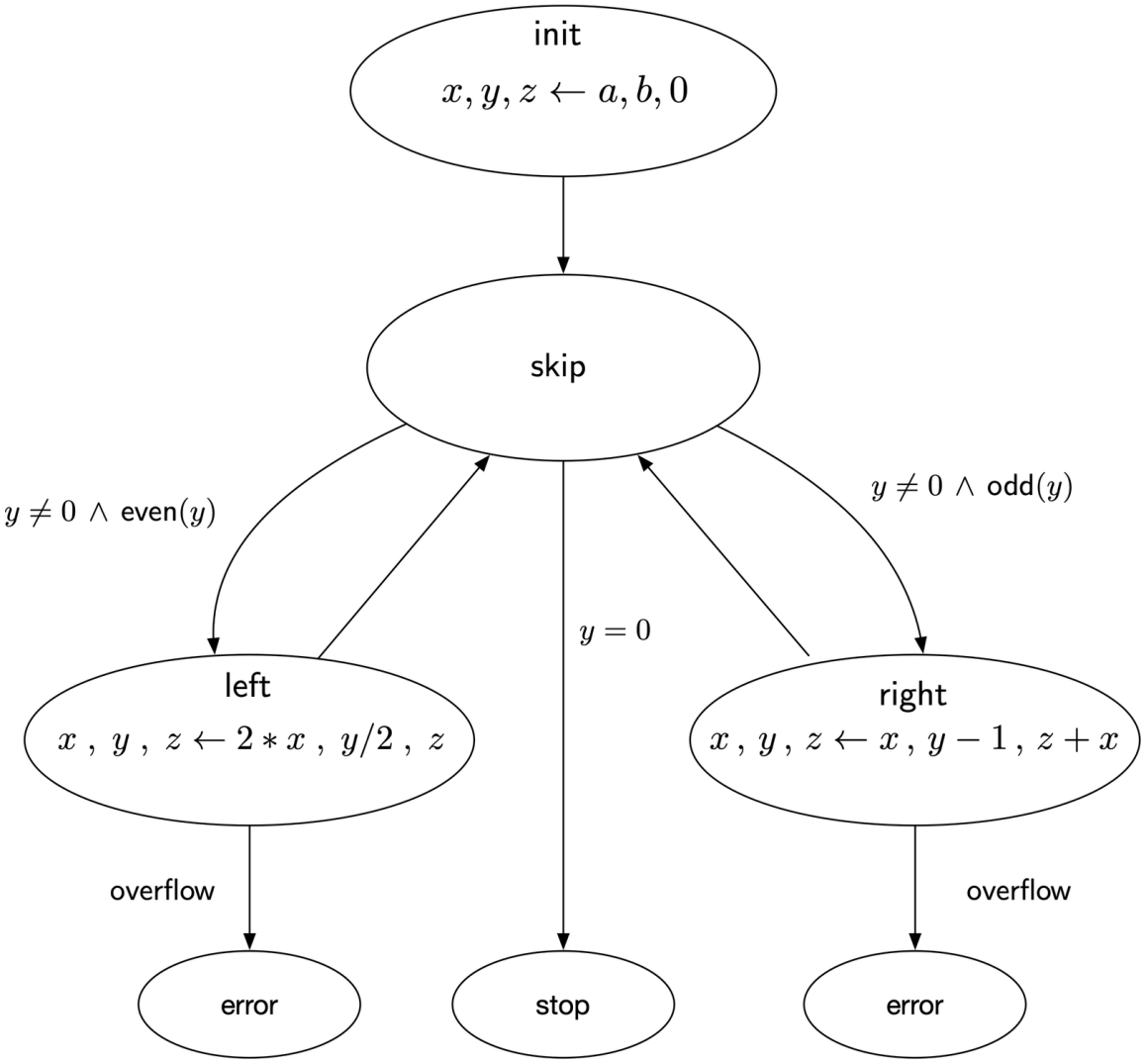
In [ ]:
```
import marimo as mo
from pysmt.shortcuts import Symbol, BV, And, Equals, get_model, Int, GE,
Or, GT, BVSGT, BVSLE, BVSLT, BVSGE, Not, BVSRem, BVLShl, BVLShr, BVSub,
BVAdd, Solver, BVURem, BVUGE, BVULT, BVMul, NotEquals, Ite, Bool, BVUGT
from pysmt.typing import BVType, INT
```

Variáveis globais

Exercício 3

Considere o problema descrito no documento +Lógica Computacional: Multiplicação de Inteiros . Nesse documento usa-se um "Control Flow Automaton" como modelo do programa imperativo que calcula a multiplicação de inteiros positivos representados por vetores de bits.



- a. Construir um SFOTS, usando BitVec's de tamanho n , que descreva o comportamento deste autómato; para isso identifique e codifique em `Z3` , as

variáveis do modelo, o estado inicial , a relação de transição e o estado de erro.

- b. Usando k-indução verifique nesse SFOTS se a propriedade $x \times y + z = a \times b$ é um invariante do seu comportamento.
- c. Usando k-indução no SFOTS acima e adicionando ao estado inicial a condição $a < 2^{n/2} \wedge b < 2^{n/2}$, verifique a segurança do programa; nomeadamente prove que, com tal estado inicial, o estado de erro nunca é acessível.
- d. Codifique a propriedade de animação que exprime que o programa termina quaisquer que sejam os valores iniciais de a e b e prove que ela se verifica no SFOTS que construiu.

Estados

```
assert a >= 0 and b >= 0
x , y, z = a , b , 0
0: while not y == 0:
1:   if even(y):
2:       x , y , z = x << 1 , y >> 1 , z
     else:
3:       x , y , z = x , y - 1, z + x
4: stop
5: erro
```

Parte a

Construção do SFOTS

Começamos pela declaração dos estados e a inicialização do primeiro estado, de acordo com as regras indicadas no início do programa.

In [ ]:
```
def declare(i):
    state = {}
    state['pc'] = Symbol('pc'+str(i),INT)
    state['x'] = Symbol('x'+str(i),BVType(N))
    state['y'] = Symbol('y'+str(i),BVType(N))
    state['z'] = Symbol('z'+str(i),BVType(N))
    state['a'] = Symbol('a'+str(i),BVType(N))
    state['b'] = Symbol('b'+str(i),BVType(N))
    return state

def init(state, i=None):
    constraints = [
        Equals(state['pc'], Int(0)),
        Equals(state['x'], state['a']),
        Equals(state['y'], state['b']),
        Equals(state['z'], BV(0, N))
    ]
```

```
    if SIGNAL:
        constraints.append(BVSGE(state['a'], BV(0,N)))
        constraints.append(BVSGE(state['b'], BV(0,N)))

    if i is not None:
        constraints.append(Equals(state['a'], i['a']))
        constraints.append(Equals(state['b'], i['b']))

    return And(constraints)
```

Transições

In [ ]:

```
def equal(curr, prox, list_symbol):
    return [Equals(prox[c], curr[c]) for c in list_symbol]

def error(state):
    return Equals(state['pc'], Int(5))

def stop(state):
    return Equals(state['pc'], Int(4))

def trans(curr,prox):
    return Or(
        # casos de erro
        And(
            Equals(curr['pc'], Int(2)),
            BVSLT(BVLShl(curr['x'], 1), BV(0,N)) if SIGNAL else
BVULT(BVLShl(curr['x'], 1), curr['x']),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])

        ),
        And(
            Equals(curr['pc'], Int(3)),
            BVSLT(BVAdd(curr['z'], curr['x']), BV(0,N)) if SIGNAL else
BVULT(BVAdd(curr['z'], curr['x']), curr['z']),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        # loop
        And(
            Equals(curr['pc'], Int(0)),
            Not(Equals(curr['y'], BV(0, N))),
            Equals(prox['pc'], Int(1)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        # caso em que y é par
        And(
            Equals(curr['pc'], Int(1)),
            Equals(BVSRem(curr['y'], BV(2, N)), BV(0, N)) if SIGNAL else
Equals(BVURem(curr['y'], BV(2, N)), BV(0, N)),
            Equals(prox['pc'], Int(2)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
```

```
        And(
            Equals(curr['pc'], Int(2)),
            Equals(prox['pc'], Int(0)),
            Not(BVSLT(BVLShl(curr['x'], 1), BV(0,N))) if SIGNAL else
Not(BVULT(BVLShl(curr['x'], 1), curr['x'])),
            Equals(prox['x'], BVLShl(curr['x'], 1)),
            Equals(prox['y'], BVLShr(curr['y'], 1)),
            *equal(curr, prox, ['z', 'a', 'b'])
        ),
        # caso em que y é ímpar
        And(
            Equals(curr['pc'], Int(1)),
            Equals(BVSRem(curr['y'], BV(2, N)), BV(1, N)) if SIGNAL else
Equals(BVURem(curr['y'], BV(2, N)), BV(1, N)),
            Equals(prox['pc'], Int(3)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            Equals(curr['pc'], Int(3)),
            Equals(prox['pc'], Int(0)),
            Not(BVSLT(BVAdd(curr['z'], curr['x']), BV(0,N))) if SIGNAL else
Not(BVULT(BVAdd(curr['z'], curr['x']), curr['z'])),
            Equals(prox['y'], BVSub(curr['y'], BV(1, N))),
            Equals(prox['z'], BVAdd(curr['z'], curr['x'])),
            *equal(curr, prox, ['x', 'a', 'b'])
        ),
        # caso de terminação
        And(
            Equals(curr['pc'], Int(0)),
            Equals(curr['y'], BV(0, N)),
            Equals(prox['pc'], Int(4)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            Equals(curr['pc'], Int(4)),
            Equals(prox['pc'], Int(4)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            error(curr),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        )
    )
```

Gera traço

In [ ]:

```
def gera_traco(declare,init,trans,initial, kmax=300):

    with Solver(name="z3") as s:

        # states = [declare(i) for i in range(k)]
        states = [declare(0)]
        s.add_assertion(init(states[0], initial))
```

```
        for c in range(kmax):
            s.push()
            s.add_assertion(Or(Equals(states[-1]['pc'], Int(4)),
Equals(states[-1]['pc'], Int(5))))


            if s.solve():
                model = s.get_model()
                w = len(str(model.get_value(states[-1]
['x'])).constant_value()))
                a_val = model.get_value(states[0]['a']).constant_value()
                b_val = model.get_value(states[0]['b']).constant_value()
                for i, st in enumerate(states):
                    pc_val = model.get_value(st['pc']).constant_value()
                    x_val = model.get_value(st['x']).constant_value()
                    y_val = model.get_value(st['y']).constant_value()
                    z_val = model.get_value(st['z']).constant_value()
                    inv = 2 * a_val * b_val - x_val + y_val + pcA[pc_val] -
z_val
                    print(f"Estado {i:>3} : pc = {pc_val:>1}, x = {x_val:>
{w}}, y = {y_val:>{w}}, z = {z_val:>{w}}, inv = {inv:>{w}}")
                s.pop()
                return

            s.pop()
            states.append(declare(c+1))
            s.add_assertion(trans(states[c], states[c+1]))

        print("Não foi encontrado um traço válido")
```

Teste

In [ ]:
```
i = {'pc': Int(0), 'a' : BV(0, N), 'b' : BV(8, N)}
gera_traco(declare,init,trans,i)
```

```
Estado   0 : pc = 0, x = 0, y = 8, z = 0, inv = 12
Estado   1 : pc = 1, x = 0, y = 8, z = 0, inv = 11
Estado   2 : pc = 2, x = 0, y = 8, z = 0, inv = 10
Estado   3 : pc = 0, x = 0, y = 4, z = 0, inv = 8
Estado   4 : pc = 1, x = 0, y = 4, z = 0, inv = 7
Estado   5 : pc = 2, x = 0, y = 4, z = 0, inv = 6
Estado   6 : pc = 0, x = 0, y = 2, z = 0, inv = 6
Estado   7 : pc = 1, x = 0, y = 2, z = 0, inv = 5
Estado   8 : pc = 2, x = 0, y = 2, z = 0, inv = 4
Estado   9 : pc = 0, x = 0, y = 1, z = 0, inv = 5
Estado  10 : pc = 1, x = 0, y = 1, z = 0, inv = 4
Estado  11 : pc = 3, x = 0, y = 1, z = 0, inv = 2
Estado  12 : pc = 0, x = 0, y = 0, z = 0, inv = 4
Estado  13 : pc = 4, x = 0, y = 0, z = 0, inv = 0
```

Parte b

Indução $x \times y + z = a \times b$

In [ ]:

```
def k_inducao(declare,init,trans,inv,K):
    for k in range(1, K):
        with Solver(name="z3") as solver:
            states = [declare(i) for i in range(k)]
            base = [init(states[0])]
            for i in range(k-1):
                base.append(trans(states[i], states[i+1]))

            caso_base = And(And(base), Or([Not(inv(s)) for s in states]))
            solver.add_assertion(caso_base)

            if solver.solve():
                print(f"Falha caso base com k = {k}")
                model = solver.get_model()
                w = len(str(model.get_value(states[-1]
['x']).constant_value()))
                a_val = model.get_value(states[0]['a']).constant_value()
                b_val = model.get_value(states[0]['b']).constant_value()
                for i, st in enumerate(states):
                    pc_val = model.get_value(st['pc']).constant_value()
                    x_val = model.get_value(st['x']).constant_value()
                    y_val = model.get_value(st['y']).constant_value()
                    z_val = model.get_value(st['z']).constant_value()
                    inv = 2 * a_val * b_val - x_val + y_val + pcA[pc_val] -
z_val
                    print(f"Estado {i:>3} : pc = {pc_val:>1}, x = {x_val:>
{w}}, y = {y_val:>{w}}, z = {z_val:>{w}}, inv = {inv:>{w}}")
                return False
            solver.reset_assertions()
            states = [declare(i) for i in range(k+1)]
            inductive = []
            for i in range(k):
                inductive.append(inv(states[i]))
                inductive.append(trans(states[i], states[i+1]))

            inductive.append(Not(inv(states[k])))
            step_case = And(inductive)

            solver.add_assertion(step_case)

            if solver.solve():
                print(f"Falha caso indutivo k = {k}")

            else:
                print(f"Invariante válido para k = {k}")
                return True

    return False



def inv(state):
```

```
    return Equals(BVAdd(BVMul(state['x'], state['y']), state['z']),
BVMul(state['a'], state['b']))
```

Teste

In [ ]:
```
k_inducao(declare,init,trans,inv,10)
```

```
Falha caso indutivo k = 1
Invariante válido para k = 2
```
True

Parte c

k-indução com a condição $a < 2^{n/2} \wedge b < 2^{n/2}$ da propriedade NotEquals(state['pc'], Int(5))

Teste para mostrar a inseguraça do programa antes da restrição.

In [ ]:
```
def noterror(state):
    return NotEquals(state['pc'], Int(5))

k_inducao(declare,init,trans,noterror,10)
```

```
Falha caso indutivo k = 1
Falha caso indutivo k = 2
Falha caso indutivo k = 3
Falha caso base com k = 4
Estado   0 : pc = 0, x = 156, y =   4, z =   0, inv = 1100
Estado   1 : pc = 1, x = 156, y =   4, z =   0, inv = 1099
Estado   2 : pc = 2, x = 156, y =   4, z =   0, inv = 1098
Estado   3 : pc = 5, x = 156, y =   4, z =   0, inv = 1095
```
False

Função init com a condição $a < 2^{n/2} \wedge b < 2^{n/2}$

In [ ]:
```
def init2(state, i=None):
    constraints = [
        Equals(state['pc'], Int(0)),
        Equals(state['x'], state['a']),
        Equals(state['y'], state['b']),
        Equals(state['z'], BV(0, N))
    ]

    if SIGNAL:
        constraints.append(BVSGE(state['a'], BV(0,N)))
        constraints.append(BVSGE(state['b'], BV(0,N)))
```

```
        constraints.append(BVSLT(state['a'], BV(2**(N/2),N)))
        constraints.append(BVSLT(state['b'], BV(2**(N/2),N)))

    else:
        constraints.append(BVULT(state['a'], BV(2**int(N/2),N)))
        constraints.append(BVULT(state['b'], BV(2**int(N/2),N)))

    if i is not None:
        constraints.append(Equals(state['a'], i['a']))
        constraints.append(Equals(state['b'], i['b']))

    return And(constraints)
```

Teste com a condição

In [ ]:
```
# k_inducao2(noterror, 100)

k_inducao(declare,init2,trans,noterror, 100)
```

```
 Falha caso indutivo k = 1
 Falha caso indutivo k = 2
 Falha caso indutivo k = 3
 Falha caso indutivo k = 4
 Falha caso indutivo k = 5
 Falha caso indutivo k = 6
 Falha caso indutivo k = 7
 Falha caso indutivo k = 8
 Falha caso indutivo k = 9
 Falha caso indutivo k = 10
 Falha caso indutivo k = 11
 Falha caso indutivo k = 12
 Falha caso indutivo k = 13
 Falha caso indutivo k = 14
 Falha caso indutivo k = 15
 Falha caso indutivo k = 16
 Falha caso indutivo k = 17
 Falha caso indutivo k = 18
 Falha caso indutivo k = 19
 Falha caso indutivo k = 20
 Falha caso indutivo k = 21
 Falha caso indutivo k = 22
 Falha caso indutivo k = 23
 Falha caso indutivo k = 24
 Falha caso indutivo k = 25
 Falha caso indutivo k = 26
 Falha caso indutivo k = 27
 Falha caso indutivo k = 28
 Falha caso indutivo k = 29
 Falha caso indutivo k = 30
 Falha caso indutivo k = 31
 Falha caso indutivo k = 32
 Falha caso indutivo k = 33
 Falha caso indutivo k = 34
 Falha caso indutivo k = 35
 Falha caso indutivo k = 36
 Falha caso indutivo k = 37
 Falha caso indutivo k = 38
 Falha caso indutivo k = 39
 Falha caso indutivo k = 40
 Falha caso indutivo k = 41
 Falha caso indutivo k = 42
 Falha caso indutivo k = 43
 Falha caso indutivo k = 44
 Falha caso indutivo k = 45
 Falha caso indutivo k = 46
 Invariante válido para k = 47
True
```

Parte d

Propriedade de animação

In [ ]:
```
def animacao(declare, init, trans, P_estrela, K=10):
    for k in range(1, K + 1):
        s = [declare(i) for i in range(k + 2)]

        with Solver(name="z3") as solver:
            solver.add_assertion(init(s[0]))
            for i in range(k):
                solver.add_assertion(trans(s[i], s[i + 1]))
            solver.add_assertion(Not(And([P_estrela(s[i], s[i + 1]) for i
in range(k)])))
            if solver.solve():
                print("Falha no caso base")
                return False

            solver.reset_assertions()

            for i in range(k):
                solver.add_assertion(P_estrela(s[i], s[i + 1]))
            for i in range(k + 1):
                solver.add_assertion(trans(s[i], s[i + 1]))
            solver.add_assertion(Not(P_estrela(s[k], s[k + 1])))
            if not solver.solve():
                print(f"Provado com k={k}")
                return True
            else:
                print(f"Falha caso indutivo com k = {k}")


    print(f"Inconclusivo até {K}")
    return None
```

Teste

In [ ]:
```
def P_estrela(curr, prox):
    y_desce   = BVULT(prox['y'], curr['y'])
    y_igual   = Equals(prox['y'], curr['y'])
    pc_avanca = GT(prox['pc'], curr['pc'])
    parado    = And(Equals(prox['pc'], curr['pc']), GE(curr['pc'], Int(4)))
    return Or(y_desce, And(y_igual, pc_avanca), And(y_igual, parado))

animacao(declare, init, trans, P_estrela)
```

```
Falha caso indutivo com k = 1
Provado com k=2
```

True

Uso de LLM

Neste trabalho, usamos o Claude como apoio neste trabalho. Em concreto:

- Explicação de funções relacinais para BitVecs, com e sem sinal;

- Debug de código: BV(1,N) trocado com Int(1), índices e chaves erradas em listas e dicionários, k-indução(parte b), função gera_traco, etc;
- Explicação do push e pop do solver e modificação função gera_traco com push e pop.

Link para o chat: https://claude.ai/share/ffbc0723-fcc8-422f-8036-22b39d82a36d